

THE WHITE HOUSE LOGS

TURNING TRANSPARENCY INTO UNDERSTANDING



NICHOLAS RORIGUEZ | 12/20/25

Purpose

The White House has been kind enough in previous years to provide the comings and goings of its premises in the form of visitor logs or "...WAVES-ACCESS-RECORDS". The White House releases this information as a form of transparency, but this does not equate to the general public understanding this information. Just the month of December 2023 contains 122,138 accesses to the White House, and these accesses are not always single people; they may be groups of people in the 100's (2023.12_WAVES-ACCESS-RECORDS).

My call to action is to turn this inordinate amount of information into something the general public can use.

This will be done by analyzing patterns of entrants, separating the meaningful from the noise, to help the general public see how people flow through the White House. This review invites and will empower all members of the general public, whether students, working professionals, or engaged citizens, to explore these visitor logs with a purpose.

Audience

According to the National Center for Education Statistics, as of 2022, 95% of adults aged 25 to 29 have completed high school. This percentage falls drastically to 49% of adults aged 25 to 29 who have completed an associate's degree or higher (National Center for Education Statistics). Due to education levels, this information will be presented at a high school level and to the general public. Someone with a general understanding of high school-level math, not a direct stakeholder of any particular political process, likely to be unfamiliar with the data publicly provided by the White House, think people who might be your mother, brother, or daughter.

Gestalts Principles

The going principle is that individuals will perceive visual information as patterns and as a whole, not as individual pieces. With this in mind, my data visualizations will lend to the psychological thought that the viewer will automatically interpret the information given as a group. This empowers me to intentionally shape how the general public will read my charts.

Data Source

Data, as previously mentioned, have been graciously provided by the United States Government and were previously locatable at <https://www.whitehouse.gov/disclosures/visitor-logs/>. Unfortunately, our current administration does not see the value in keeping the transparency of previous administrations, citing privacy and security concerns (US Dataverse). This had led to the page no longer in operation.

Design

I will be working with the following variables or pieces of information (U.S. General Services Administration):

UIN (Unique Identification Number); the primary key for identifying a visitor

Last Name + First Name; lends to human-readable charts, connects with the general public, and is helpful if the UIN is missing

Meeting Location; will tell the audience where a visitor is going to signal the purpose of the trip

Meeting Room; will give insight that can be immediately understood and adds nuance to Meeting Location

The combination of these pieces of information will tell a story about visitors, how often they are visiting, and offer a deeper transparency of the same information provided.

Ethical Considerations

Preparing this dataset for analysis required I perform standard cleaning steps such as parsing dates, normalizing column names, removing rows with missing fields, and aggregating visits by day and location; in doing so, no records were deleted without explicit notation of the criteria used. Because the White House Logs are public records, they fall under legal frameworks such as the Freedom of Information Act and the White House's own visitor log disclosure policies, meaning the data are legally accessible. The primary risks introduced by the transformation and visualization of this data involve unintentionally overstating patterns, or implying causation

where there is only correlation, because of this care was taken to label all filters used, explain each transformation, and avoiding of speculative interpretations.

However, some assumptions were necessary, such as treating acronyms as meeting location identifiers and grouping visits by date, but these are notated, and no data were excluded without explanation. The dataset originates from an official government release, which provides a high level of credibility, political positions aside, and it was acquired ethically through a publicly available channel intended for purposes like public transparency, and how I used it. To mitigate ethical concerns, I focused on aggregate patterns rather than individual behaviors, avoided identifying or profiling specific visitors, documented all transformations, and emphasized that visualizations are descriptions rather than definitive statements about a political intent or policy.

References:

National Center for Education Statistics. *Fast Facts: Educational Attainment*. U.S. Department of Education, <https://nces.ed.gov/fastfacts/display.asp?id=27>. Accessed 20 Dec. 2025.

US Dataverse. Visitor Access Records. <https://www.usdataverse.com/en/data/m/government/Visitor-Access-Records.html>. Accessed 20 Dec. 2025.

U.S. General Services Administration. *GSA.gov*. <https://www.gsa.gov/>. Accessed 26 Dec. 2025.

Data Source:

The White House. *Visitor Logs*. <https://www.whitehouse.gov/disclosures/visitor-logs/>. Accessed 20 Dec. 2025.

2023.01_WAVES-ACCESS-RECORDS

2023.02_WAVES-ACCESS-RECORDS

2023.03_WAVES-ACCESS-RECORDS

2023.04_WAVES-ACCESS-RECORDS

2023.05_WAVES-ACCESS-RECORDS

2023.06_WAVES-ACCESS-RECORDS

2023.07_WAVES-ACCESS-RECORDS

2023.08_WAVES-ACCESS-RECORDS

2023.09_WAVES-ACCESS-RECORDS

2023.10_WAVES-ACCESS-RECORDS

2023.11_WAVES-ACCESS-RECORDS

Nicholas Rodriguez DSC 640 Week 3 & 4 Assignment - White House Logs

April 9, 2026

This project intends to interpret the Waves Access Records provided by the United States White House. Providing information is great for transparency, but without understanding the information is useless.

25 .csv datasets were provided, of which 12 will be used. The 12 selected are the most recent and comprehensive, since they cover the entirety of the year 2023. ***

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

sns.set(style="whitegrid")

# Current directory
data_dir = Path(".")

# This Path will pick up the files:
# 2023.01_WAVES-ACCESS-RECORDS.csv ... 2023.12_WAVES-ACCESS-RECORDS.csv

files = sorted(data_dir.glob("2023.*_WAVES-ACCESS-RECORDS*.csv"))

print("Files found:", len(files))
for f in files:
    print(" -", f.name)

dfs = []
for f in files:
    df = pd.read_csv(f, dtype=str) # Read as strings first

    df["source_file"] = f.name
    dfs.append(df)

full = pd.concat(dfs, ignore_index=True)
```

Files found: 12

- 2023.01_WAVES-ACCESS-RECORDS.csv
- 2023.02_WAVES-ACCESS-RECORDS.csv
- 2023.03_WAVES-ACCESS-RECORDS.csv
- 2023.04_WAVES-ACCESS-RECORDS.csv
- 2023.05_WAVES-ACCESS-RECORDS.csv
- 2023.06_WAVES-ACCESS-RECORDS.csv
- 2023.07_WAVES-ACCESS-RECORDS.csv
- 2023.08_WAVES-ACCESS-RECORDS.csv
- 2023.09_WAVES-ACCESS-RECORDS.csv
- 2023.10_WAVES-ACCESS-RECORDS.csv
- 2023.11_WAVES-ACCESS-RECORDS.csv
- 2023.12_WAVES-ACCESS-RECORDS.csv

```
[2]: ###
# Basic Cleaning
###

# 1. Convert datetime columns where date and time are already combined

datetime_cols = [
    "Appointment Made Date",
    "Appointment Start Date",
    "Appointment End Date"
]
for col in datetime_cols:
    full[col] = pd.to_datetime(full[col], errors="coerce")

# 2. Extract time components for analysis

full["year"] = full["Appointment Start Date"].dt.year
full["month"] = full["Appointment Start Date"].dt.to_period("M")

# 3. Identify recurring visitors using UIN

full["UIN"] = full["UIN"].astype(str)

visit_counts = (
    full.groupby("UIN")
    .size()
    .rename("visit_count")
)

full = full.join(visit_counts, on="UIN")
full["is_recurring"] = full["visit_count"] > 1

# Subset for recurring visitors
```



```

recurring = full[full["is_recurring"]].copy()

# 4. Check

print("Rows loaded:", len(full))
print("Unique visitors:", full["UIN"].nunique())
print("Recurring visitors:", recurring["UIN"].nunique())

```

Rows loaded: 843216
 Unique visitors: 99352
 Recurring visitors: 95646

```

[3]: ###
# 1. Bar Chart of the Top 10 Recurring Visitors by Number of Visits
###

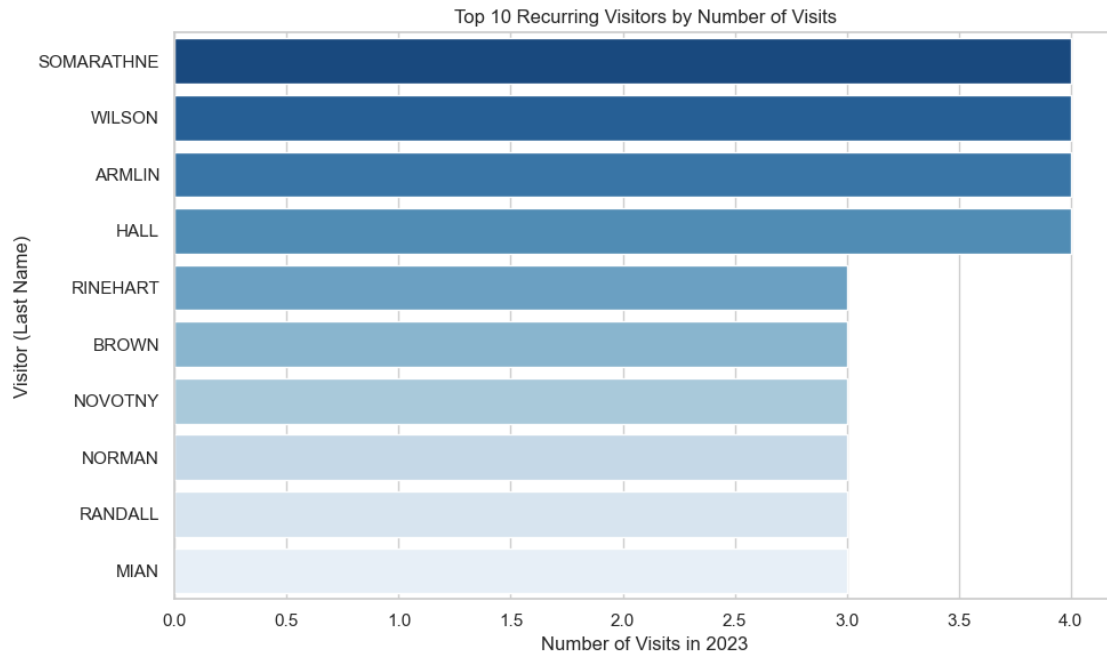
# Aggregate visits for recurring visitors

top_recurring = (
    recurring
    .groupby(["UIN", "Last Name", "First Name"])["Appointment Start Date"]
    .count()
    .reset_index(name="num_visits")
    .sort_values("num_visits", ascending=False)
    .head(10)
)

plt.figure(figsize=(10, 6))
sns.barplot(
    data=top_recurring,
    x="num_visits",
    y="Last Name",
    hue="Last Name",
    palette="Blues_r"
)

plt.xlabel("Number of Visits in 2023")
plt.ylabel("Visitor (Last Name)")
plt.title("Top 10 Recurring Visitors by Number of Visits")
plt.tight_layout()
plt.show()

```



```
[4]: ###
# 2. Column Chart of One-Time vs. Recurring Visitors
###

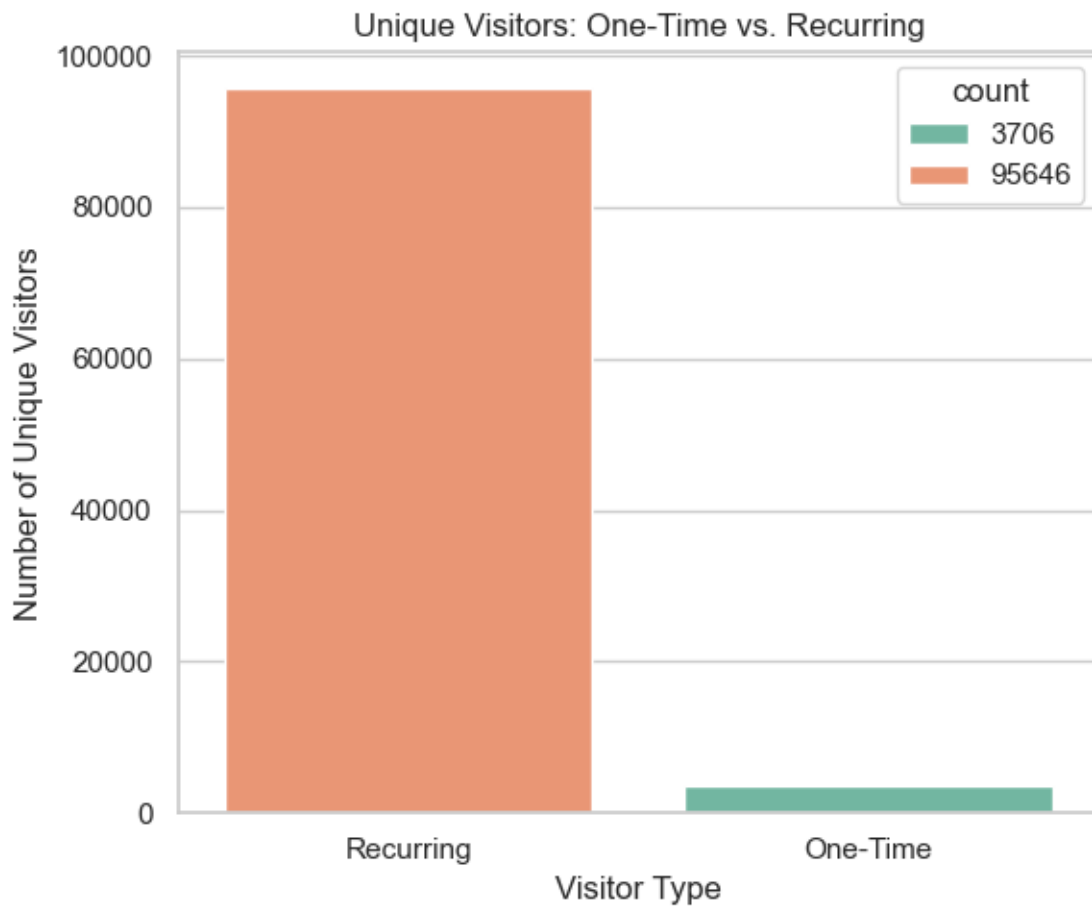
visitor_status = (
    full
    .drop_duplicates("UIN")["UIN", "is_recurring"]
    .assign(status=lambda d: d["is_recurring"].map({True: "Recurring", False: "One-Time"}))
)

status_counts = visitor_status["status"].value_counts().reset_index()
status_counts.columns = ["status", "count"]

plt.figure(figsize=(6, 5))
sns.barplot(
    data=status_counts,
    x="status",
    y="count",
    hue="count",
    palette="Set2"
)

plt.xlabel("Visitor Type")
plt.ylabel("Number of Unique Visitors")
```

```
plt.title("Unique Visitors: One-Time vs. Recurring")
plt.tight_layout()
plt.show()
```



```
[5]: ###
# 3. Stacked Bars with Time as Monthly Recurring Visits by Meeting Location
###

# Map acronyms to full names

location_map = {
    "OEOB": "Old Executive Office Building",
    "EEOB": "Eisenhower Executive Office Building",
    "WH": "White House",
    "ITR": "In-Town Residence",
    "VPR": "Vice President's Residence",
    "NEOB": "New Executive Office Building"
```

```

}

# Apply mapping with fallback to original acronym

recurring["Location Full Name"] = (
    recurring["Meeting Location"]
    .map(location_map)
    .fillna(recurring["Meeting Location"])
)

# Use the full name column for grouping

location_col = "Location Full Name"

monthly_loc = (
    recurring
    .dropna(subset=["Appointment Start Date"])
    .groupby([recurring["Appointment Start Date"].dt.to_period("M"),
    ↪location_col])
    .size()
    .rename("visits")
    .reset_index()
)

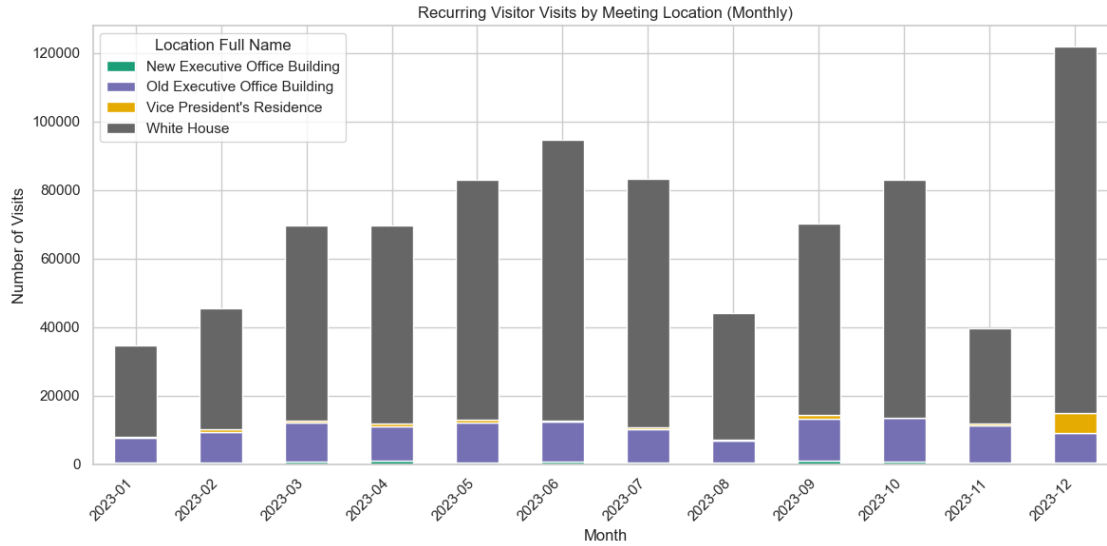
monthly_loc["month"] = monthly_loc["Appointment Start Date"].astype(str)

pivot_loc = monthly_loc.pivot(index="month", columns=location_col,
    ↪values="visits").fillna(0)

pivot_loc.plot(
    kind="bar",
    stacked=True,
    figsize=(12,6),
    colormap="Dark2"
)

plt.xlabel("Month")
plt.ylabel("Number of Visits")
plt.title("Recurring Visitor Visits by Meeting Location (Monthly)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```



```
[6]: ###
# 4. Scatterplot with Time as Daily Recurring Visits by Location
###

# List of all locations to visit

all_locations = recurring["Meeting Location"].dropna().unique()

# Map acronyms to full names

location_map = {
    "OEOB": "Old Executive Office Building",
    "EEOB": "Eisenhower Executive Office Building",
    "WH": "White House",
    "ITR": "In-Town Residence",
    "VPR": "Vice President's Residence",
    "NEOB": "New Executive Office Building"
}

recurring["Location Full Name"] = (
    recurring["Meeting Location"]
    .map(location_map)
    .fillna(recurring["Meeting Location"])
)

# Prepare the data

scatter_df = (
```

```

    recurring
    .dropna(subset=["Appointment Start Date", "Location Full Name"])
    .copy()
)

scatter_df["date"] = scatter_df["Appointment Start Date"].dt.date

# scatter_df["Location Full Name"] = (
#     scatter_df["Meeting Location"]
#     .map(location_map)
#     .fillna(scatter_df["Meeting Location"])
# )

# Aggregate visits per day per location

daily_counts = (
    scatter_df
    .groupby(["date", "Location Full Name"])
    .size()
    .reset_index(name="num_visits")
)

# Plot with Facetgrid, one panel per location

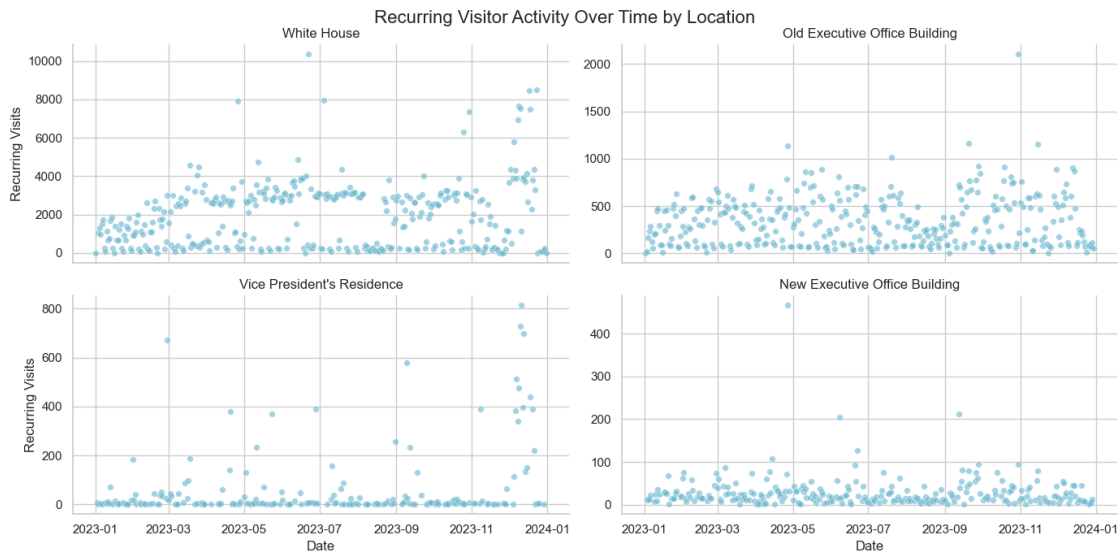
g = sns.FacetGrid(
    daily_counts,
    col="Location Full Name",
    col_wrap=2,
    height=3.5,
    aspect=2,
    sharey=False
)

g.map_dataframe(
    sns.scatterplot,
    x="date",
    y="num_visits",
    alpha=0.6,
    s=30,
    color="c"
)

g.set_axis_labels("Date", "Recurring Visits")
g.set_titles("{col_name}")
g.fig.suptitle("Recurring Visitor Activity Over Time by Location", fontsize=16)
g.fig.tight_layout()
g.fig.subplots_adjust(top=0.92) # Spacing for title

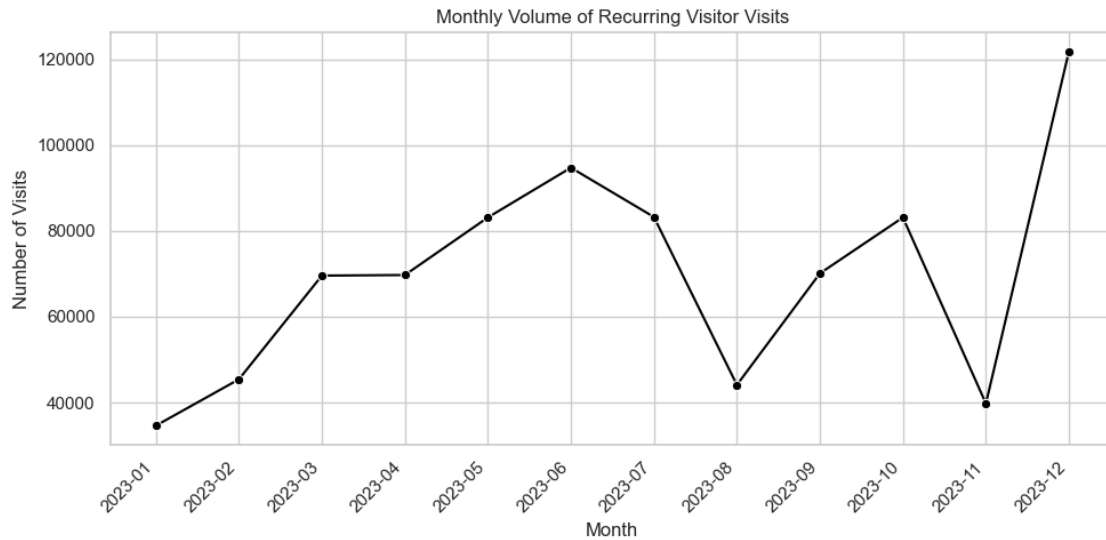
```

```
plt.show()
```



```
[7]: ###  
# 5. Line Chart of Monthly Volume of Recurring Visitor Visits  
###  
  
monthly_visits = (  
    recurring  
    .dropna(subset=["Appointment Start Date"])  
    .groupby(recurring["Appointment Start Date"].dt.to_period("M"))  
    .size()  
    .rename("num_visits")  
    .reset_index()  
)  
  
monthly_visits["month"] = monthly_visits["Appointment Start Date"].astype(str)  
  
plt.figure(figsize=(10, 5))  
sns.lineplot(  
    data=monthly_visits,  
    x="month",  
    y="num_visits",  
    marker="o",  
    color="black"  
)  
  
plt.xlabel("Month")  
plt.ylabel("Number of Visits")
```

```
plt.title("Monthly Volume of Recurring Visitor Visits")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()
```



```
[8]: ###
# 6. Step Chart of Cumulative Distinct Recurring Visitors Over Time
###

first_visits = (
    recurring
    .dropna(subset=["Appointment Start Date"])
    .groupby("UIN")["Appointment Start Date"]
    .min()
    .sort_values()
    .reset_index(name="first_visit_dt")
)

cum_df = (
    first_visits
    .groupby(first_visits["first_visit_dt"].dt.date)
    .size()
    .rename("new_recurring")
    .reset_index()
)

cum_df["cum_recurring"] = cum_df["new_recurring"].cumsum()
```



```

plt.figure(figsize=(10, 5))
plt.step(
    cum_df["first_visit_dt"],
    cum_df["cum_recurring"],
    where="post",
    color="black"
)

plt.xlabel("Date")
plt.ylabel("Cumulative Number of Recurring Visitors")
plt.title("Cumulative Distinct Recurring Visitors For 2023")
plt.tight_layout()
plt.show()

```

